23. Authentication and Authorization

▼ Authentication & Authorization

✈️ **Airport Example**

- First, you show your **ID** at security

  👉 They check **who you are**

  👉 This is **Authentication**

- Then, at the gate, they check your **boarding pass**

  👉 They check **where you are allowed to go**

  👉 This is **Authorization**

👉 **Remember this line (important):**

- **Authentication = Who are you?**
- **Authorization = What can you do?**

▼ 1. What is `Authentication`?

- Authentication is the process of **verifying the identity** of a user or system.
- In simple words:
- 👉 **Checking whether you are a real user or not**
- **Example →** When you log in using **email & password**, the system checks:
  - Is this email registered?
  - Is the password correct?
  - If yes → ✅ You are **authenticated**

- **Common Authentication Methods:**
  1. Username + Password

2. OTP (One-Time Password)
3. Biometrics (fingerprint, Face ID)
4. Social Login (Google, Facebook, etc)
5. Token-based (like JWT)

▼ 2. What is `Authorization`?

- Authorization means checking what the user is allowed to access, after the authentication.
- In simple words:

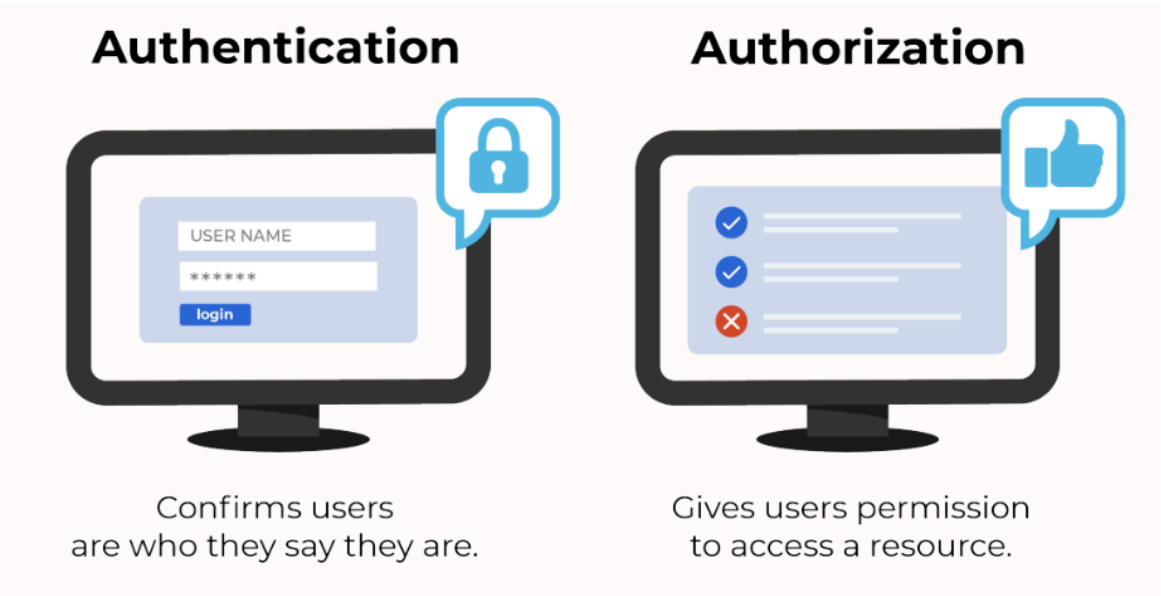
  👉 **Checking permissions**
- **Example →**
  - Normal user → Can see **only their profile**
  - Admin user → Can see **all users, delete data, edit data**

  👉 Even if you are logged in,

  ❌ you **cannot access admin pages** if you are not authorized.

▼ Authentication vs Authorization

- In short, access to a resource is protected by both authentication and authorization. If you can't prove your identity, you won't be allowed into a resource. And even if you can prove your identity, if you are not authorized for that resource, you will still be denied access.



- **Here's a quick overview of the differences between authentication and authorization:**

Authentication	Authorization
Determines whether users are who they claim to be	Determines what users can and cannot access
Challenges the user to validate credentials (for example, through passwords, answers to security questions, or facial recognition)	Verifies whether access is allowed through policies and rules
Usually done before authorization	Usually done after successful authentication
Generally, transmits info through an ID Token	Generally, transmits info through an Access Token
Generally governed by the OpenID Connect (OIDC) protocol	Generally governed by the OAuth 2.0 framework
Example: Employees in a company are required to authenticate through the network before accessing their company email	Example: After an employee successfully authenticates, the system determines what information the employees are allowed to access

	Authentication	Authorization
What does it do?	Verifies credentials	Grants or denies permissions
How does it work?	Through passwords, biometrics, one-time pins, or apps	Through settings maintained by security teams
Is it visible to the user?	Yes	No
It is changeable by the user?	Partially	No
How does data move?	Through ID tokens	Through access tokens

▼ Authentication with `JWT`

- Basically, these things are done by the backend developer.
- So as a frontend developer we can use API instead of writing backend logic.
- Let's consider, we are have an API → https://api.escuelajs.co/api/v1/auth/login

Request

```
POST https://api.escuelajs.co/api/v1/auth/login
Content-Type: application/json

{
  "email": "john@mail.com",
  "password": "changeme"
}
```

- Upon successful authentication, the server returns both access and refresh tokens:


```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjE
sImlhdCI6MTY3Mjc2NjAyOCwiZXhwIjoxNjc0NDk0MDI4fQ.kCak9sLJr74
```

```
frSRVQp0_27BY4iBCgQSmoT3vQVWKzJg",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImlhdCI6MTY3Mjc2NjAyOCwiZXhwIjoxNjcyODAyMDI4fQ.P1_rB3hJ5afwiG4TWXLq6jOAcVJkvQZ2Z-ZZOnQ1dZw"
}
```


Response

Upon successful authentication, the server returns both access and refresh tokens:


```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImlhdCI6MTY3Mjc2NjAyOCw
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImlhdCI6MTY3Mjc2NjAyOC
}
```

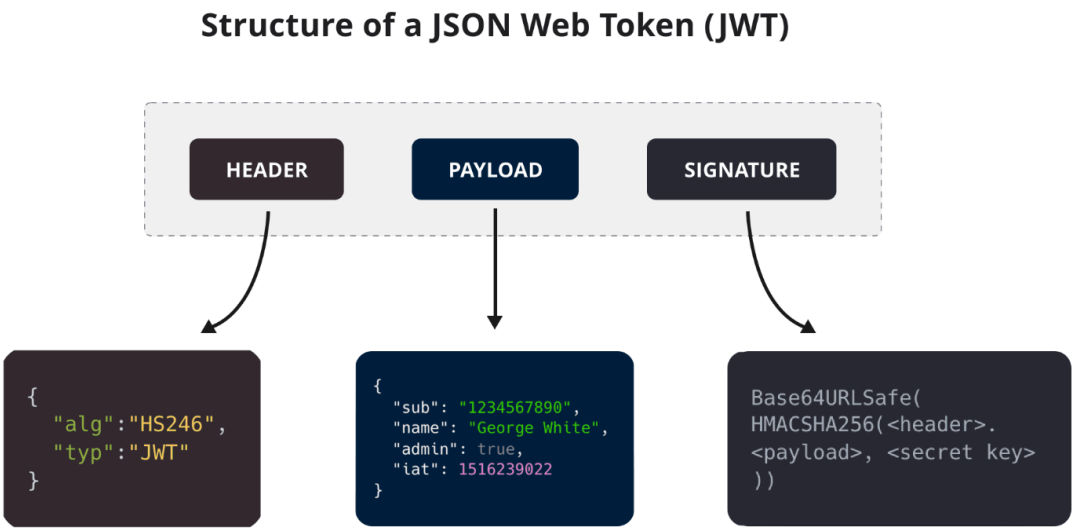

Note: The access token is valid for 20 days, and the refresh token is valid for 10 hours.

▼ What is `JWT`?

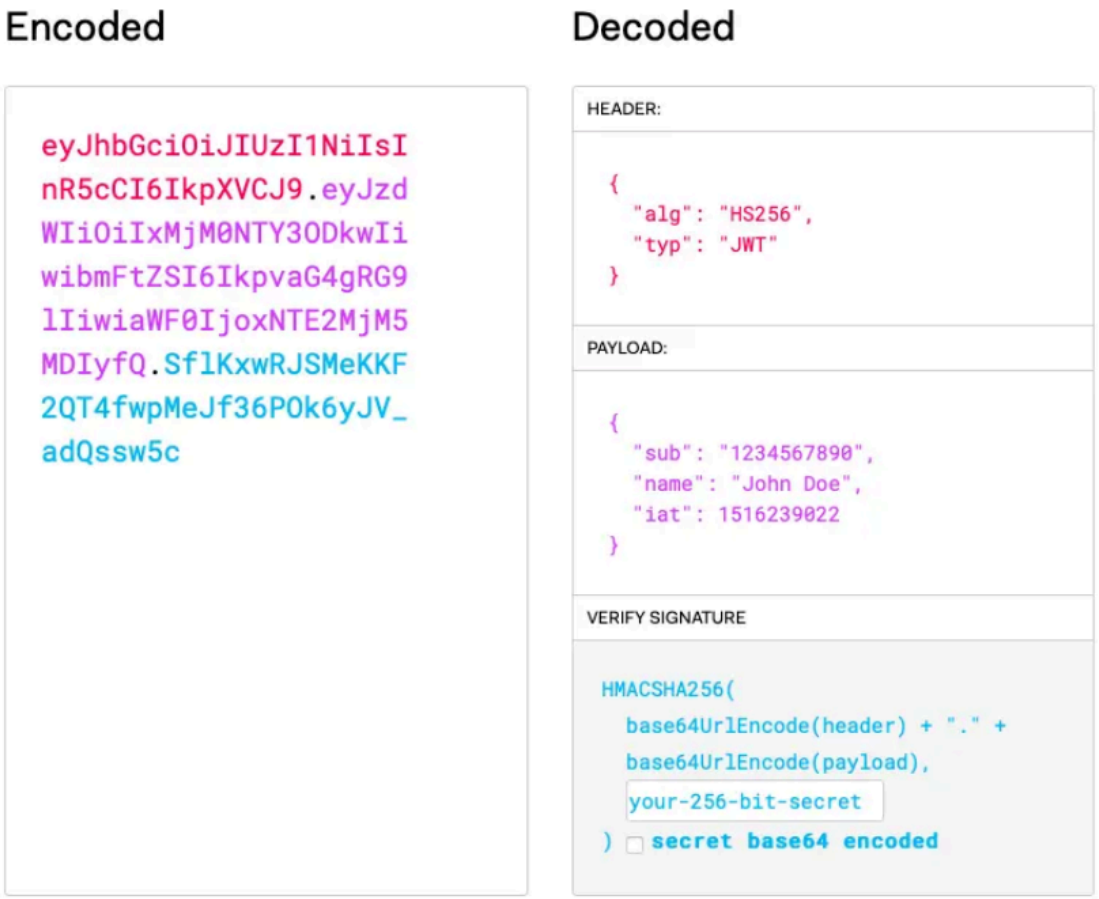
- **JWT → JSON Web Token**
- JWT is a secure and compact way to transmit information between two parties (usually a client and a server) as JSON object.
- It is commonly used for authentication and authorization in web applications.

▼ Structure of JWT?

- In its compact form, JSON Web Tokens consist of three parts separated by dots (`.`), which are:



- The following shows a JWT that has the previous header and payload encoded, and it is signed with a secret.



▼ Simple Authentication Flow

Explain this slowly 👇

1. User enters **email & password**
2. Data is sent to **API**
3. API checks credentials
4. If correct → API sends **access token**
5. Token is saved in:
   - `useReducer state`
   - `localStorage`
6. Token is sent with every protected API request
7. If token expires → use **refresh token**

👉 **Frontend uses API, backend does the hard work**